



Auditoría Automatizada de Configuraciones Inseguras con Guías de Explotación y Remediación Asistidas por IA



Realizado por Iván Seco Martín Fecha: 06/11/2025



Descripción General

Este proyecto tiene como finalidad desarrollar una herramienta capaz de auditar de forma automatizada configuraciones inseguras en sistemas Linux. A través de la ejecución de módulos inteligentes, la aplicación identifica fallos de seguridad en servicios, permisos y políticas del sistema, ofreciendo además **evidencia demostrable (PoC)** y **guías de corrección automatizadas** mediante scripts o playbooks.

El proyecto está pensado para ser **modular, seguro y replicable**, integrando tanto la perspectiva del **auditor** como la del **administrador de sistemas**. Su diseño permite validaciones periódicas, generación de informes profesionales y compatibilidad con entornos de laboratorio o producción controlada.



Objetivo del Proyecto

El objetivo principal es **automatizar la auditoría de configuraciones de seguridad** en sistemas Linux, minimizando errores humanos y acelerando el proceso de detección, demostración y remediación de vulnerabilidades.

Problemas que busca solventar

- Auditorías manuales lentas y propensas a omisiones.
 - Falta de trazabilidad y evidencia reproducible en las revisiones.
 - Ausencia de guías prácticas y scripts listos para aplicar correcciones.
 - Dificultad para mantener un control continuo sobre los cambios en configuración.
-



Valor diferencial respecto a lo que ya existe

A diferencia de herramientas como **Lynis**, **OpenSCAP** o **Tiger**, esta aplicación incorpora:

- **Pruebas PoC seguras y reproducibles:** evidencias prácticas de cada vulnerabilidad.
- **Remediaciones automáticas y controladas** (modo dry-run / confirmación manual).
- **Generación automática de documentación** (fichas Markdown → PDF/HTML).
- **Validación antes y después de la corrección**, mostrando mejoras tangibles.
- **Diseño modular** que permite agregar nuevos checks y plantillas fácilmente.

- **Opción de integración con LLM o IA local** para redactar recomendaciones personalizadas.
-

Potencial e impacto

La aplicación tiene un gran potencial porque cubre una **necesidad real en auditorías de sistemas**:

- **Escalabilidad:** permite analizar múltiples hosts desde una sola consola.
- **Aplicabilidad:** útil para administradores, auditores y formadores en ciberseguridad.
- **Eficiencia:** reduce significativamente el tiempo de revisión y documentación.
- **Transparencia:** genera evidencias verificables y reportes claros.
- **Trazabilidad:** mantiene historial de auditorías, cambios y remediaciones.

Además, puede evolucionar hacia un **servicio continuo de compliance** o integrarse con SIEMs (como ELK o Wazuh), añadiendo correlación de eventos en tiempo real.

Herramientas y tecnologías recomendadas

- **Python 3** → núcleo del motor de auditoría.
 - **osquery / subprocess / psutil** → para inspección del sistema.
 - **Ansible** → para aplicar remediaciones idempotentes.
 - **SQLite3 / JSON** → almacenamiento de hallazgos y evidencias.
 - **Pandoc / Jinja2** → generación de informes automáticos en PDF y HTML.
 - **Docker / VirtualBox** → entornos de laboratorio y pruebas controladas.
 - **Elasticsearch + Kibana** (*opcional*) → visualización avanzada de métricas.
 - **IA opcional (Llama o GPT API)** → redacción automática de explicaciones y pasos de mitigación.
-

Funcionamiento General de la Aplicación

La herramienta sigue un flujo automatizado que comprende varias fases:

1. **Configuración inicial:** el usuario define el objetivo (host o red) y el tipo de auditoría (análisis, validación, remediación).
 2. **Rastreo del sistema:** módulos inspeccionan archivos, servicios, permisos y versiones.
 3. **Análisis:** los datos se comparan con estándares de seguridad (CIS, OWASP, etc.).
 4. **Detección:** se generan hallazgos clasificados por severidad.
 5. **Evidencia y PoC:** para cada hallazgo se produce una prueba segura y reproducible.
 6. **Remediación:** se ofrecen soluciones automatizadas o guías manuales.
 7. **Validación:** el sistema reejecuta los checks tras las correcciones.
 8. **Documentación:** se genera un informe completo con métricas, evidencias y resultados.
-

Flujo de trabajo (resumen visual)

Usuario (CLI/Web)



Configuración → Rastreo → Análisis → PoC → Remediación → Validación → Repo

Cada fase genera datos estructurados (JSON/Markdown) que pueden convertirse automáticamente en reportes PDF o integrarse con otros sistemas.

Explicación de nodos (componentes principales)

Nodo	Función	Salida
Entrada / Configuración	Define alcance y parámetros de auditoría.	config.yml
Rastreadores	Recopilan configuraciones y estados del sistema.	raw_logs/
Analizador	Evalúa y clasifica hallazgos según reglas.	findings.json
Generador de PoC	Crea guías seguras y reproducibles.	fichas/*.md
Módulo de Remediación	Aplica o sugiere correcciones automáticas.	playbooks/
Validador	Revisa la efectividad de las correcciones.	validation/
Generador de Reportes	Compila todos los datos en informes PDF/HTML.	deliverables/

Ejemplo práctico real: Auditoría automatizada de seguridad en un entorno Linux corporativo

◆ Contexto inicial

Una pequeña empresa tecnológica llamada **DataSecure S.L.** dispone de varios servidores Linux que dan servicio a sus aplicaciones internas y a su página web corporativa. Durante una revisión de seguridad, el equipo de IT detecta configuraciones inconsistentes y solicita una **auditoría completa de los sistemas** antes de exponer los servicios públicamente.

El objetivo es **detectar configuraciones inseguras y malas prácticas** de forma automatizada, sin depender exclusivamente de herramientas comerciales o revisiones manuales.

◆ Escenario técnico

- **Servidor principal:** Ubuntu Server 22.04 LTS
 - **Servicios activos:** SSH, Apache2, MySQL, cron, rsyslog
 - **Red interna:** 192.168.10.0/24
 - **Usuario auditor:** "admin-audit" con privilegios sudo
 - **Entorno:** Laboratorio controlado (máquina virtual o contenedor)
-

◆ Situación problemática

Durante meses, distintos técnicos configuraron manualmente los servidores. Esto provocó inconsistencias como:

- Acceso SSH directo con el usuario root.
- Contraseñas débiles y sin expiración.
- Firewall desactivado.
- Directorios con permisos demasiado amplios.
- Servicios corriendo con privilegios innecesarios.
- Falta de registro de logs críticos.

DataSecure necesita una solución que no solo **detecte** los fallos, sino que también **los documente, demuestre y proponga soluciones seguras y automatizadas**.

◆ Objetivo del caso

Demostrar cómo la herramienta de **Auditoría Automatizada de Configuraciones** puede:

1. Detectar de forma automática configuraciones inseguras.
 2. Generar evidencias claras y reproducibles.
 3. Ofrecer guías o scripts de remediación automáticos.
 4. Validar que las correcciones se aplicaron correctamente.
 5. Documentar todo el proceso en informes listos para entregar.
-

◆ Paso 1 — Preparación de la auditoría

El auditor inicia sesión en su entorno de trabajo y ejecuta:

```
audit-cli scan --target 192.168.10.15 --mode audit --output reports/
```

La herramienta:

- Carga los módulos de auditoría configurados en config.yml.
- Identifica que el sistema es **Ubuntu 22.04 LTS**.
- Inicializa los rastreadores: SSH, usuarios, permisos, firewall, logs, kernel, paquetes.
- Crea una sesión de auditoría: AUDIT-2025-11-09-DATASECURE.

Mientras tanto, el sistema muestra:

```
[INFO] Starting system audit on 192.168.10.15
[✓] SSH configuration analyzed
[✓] User and password policies checked
[✓] Firewall and services verified
[✓] Log management evaluated
[✓] Kernel hardening parameters reviewed
[✓] Audit complete in 78 seconds
```

◆ Paso 2 — Resultados iniciales

Tras finalizar el escaneo, se genera un archivo `findings.json` con los siguientes hallazgos:

ID	Descripción	Severidad	Evidencia
CHK-SSH-001	PermitRootLogin yes en SSH	Alta	/etc/ssh/sshd_config
CHK-FW-003	Firewall (ufw) desactivado	Media	ufw status: inactive
CHK-USR-004	Usuario "backup" sin expiración	Media	/etc/shadow
CHK-PERM-006	/var/www/html es world-writable	Alta	ls -ld /var/www/html
CHK-LOG-009	auditd no activo	Media	systemctl status auditd

La herramienta asigna prioridad de remediación y vincula cada hallazgo con una plantilla PoC y una remediación.

◆ Paso 3 — Generación automática de PoC

Para cada hallazgo, se genera una ficha Markdown explicando **qué se encontró, cómo reproducirlo y por qué es un riesgo**.

Por ejemplo, para CHK-SSH-001:

```
## PoC: CHK-SSH-001 – SSH Root Login habilitado

**Objetivo:**
Comprobar que el acceso root está permitido vía SSH.
```

****Evidencia:****

Se detectó la línea `PermitRootLogin yes` en `/etc/ssh/sshd_config`.

****Prueba reproducible:****

Ejecutar `grep -i "PermitRootLogin" /etc/ssh/sshd_config``

Resultado esperado: ``PermitRootLogin yes``

****Impacto:****

Permitir inicio de sesión root facilita ataques de fuerza bruta o explotac

****Mitigación sugerida:****

Editar el archivo y establecer ``PermitRootLogin no``.

La ficha se guarda automáticamente en `reports/192.168.10.15/fichas/CHK-SSH-001.md`.

◆ Paso 4 — Ejecución de remediaciones automáticas

María, la administradora, revisa los hallazgos desde el panel CLI o web y decide aplicar las correcciones recomendadas.

Primero, simula los cambios:

```
audit-cli remediate --target 192.168.10.15 --dry-run
```

La app muestra:

```
[DRY-RUN] Would disable root SSH login
[DRY-RUN] Would activate UFW with default deny incoming
[DRY-RUN] Would set password expiration for user 'backup'
[DRY-RUN] Would restrict permissions on /var/www/html
```

Tras revisar el resumen, ejecuta la remediación real:

```
audit-cli remediate --target 192.168.10.15 --apply
```

Internamente se aplican **playbooks Ansible** que modifican configuraciones, ajustan permisos y reinician servicios afectados.

◆ Paso 5 — Validación post-corrección

Finalizada la remediación, el auditor ejecuta:

```
audit-cli validate --target 192.168.10.15
```

El sistema:

- Repite los mismos checks.
- Compara los resultados con el estado previo.
- Marca como **mitigados** los hallazgos corregidos.

Resultado:

Hallazgo	Estado anterior	Estado actual	Resultado
SSH root login	yes	no	✅ Corregido
Firewall	inactive	active	✅ Corregido
Usuario backup	sin expiración	expira en 90 días	✅ Corregido
Permisos /var/www	777	755	✅ Corregido
auditd	inactivo	inactivo	⚠️ Pendiente (requiere paquete)

◆ Paso 6 — Generación de informes y documentación

El auditor genera el informe final:

```
audit-cli report --target 192.168.10.15 --format pdf
```

La aplicación compila todos los resultados y genera tres archivos:

1. Informe técnico detallado:

- Fichas PoC y remediaciones aplicadas.
- Evidencias (capturas, líneas de configuración, comandos).
- Comparativas antes/después.

2. Resumen ejecutivo:

- Indicadores de riesgo global.
- Gráfico de mitigación (% hallazgos corregidos).
- Recomendaciones futuras.

3. Paquete de evidencias comprimido:

- Logs, archivos de configuración y hashes de integridad.

Fragmento del resumen final:

```
Auditoría completada: 09/11/2025 14:22
Host auditado: 192.168.10.15
Hallazgos totales: 5
Mitigados: 4 (80%)
Pendientes: 1 (20%)
Nivel de riesgo actual: Bajo
Tiempo total: 2m 43s
```

◆ Paso 7 — Seguimiento periódico

Un mes después, DataSecure programa auditorías recurrentes automáticas:

```
audit-cli schedule --target 192.168.10.15 --every "30d"
```

El sistema realiza comprobaciones automáticas y alerta si reaparecen configuraciones inseguras, consolidando así un **ciclo de mejora continua**.

◆ Resultados del caso práctico

- La auditoría inicial detectó **5 configuraciones inseguras críticas**.
- En menos de **3 minutos**, se detectaron, documentaron y corrigieron automáticamente 4 de ellas.
- Se generaron informes técnicos y ejecutivos completos **sin intervención manual**.
- El sistema quedó **bastionado y documentado**, con capacidad de revisión periódica.

◆ Conclusión del caso

Este caso práctico demuestra el valor real de la herramienta:

- **Reduce drásticamente el tiempo de auditoría y documentación.**
- **Genera evidencias y reportes reproducibles y profesionales.**
- **Facilita el cumplimiento de estándares de seguridad.**
- **Permite auditorías continuas sin depender de personal experto.**

En un entorno académico o profesional, esta demostración prueba la **madurez técnica, aplicabilidad y potencial de impacto** del proyecto.

Bibliografía y referencias

- CIS Benchmarks – Center for Internet Security.
 - OWASP Foundation – Secure Configuration Guidelines.
 - Lynis Project – <https://cisofy.com/lynis/>
 - OpenSCAP Project – <https://www.open-scap.org/>
 - Red Hat Ansible Automation Platform.
 - MITRE ATT&CK Framework – <https://attack.mitre.org/>
 - NIST SP 800-123 – Guide to General Server Security.
-



Posibles ampliaciones futuras

- **Integración con SIEM (Wazuh, ELK)** para correlación de eventos.
 - **Dashboard web interactivo** con métricas de riesgo.
 - **Modo continuo (daemon)** para comprobación periódica.
 - **Clasificación inteligente** de hallazgos con Machine Learning.
 - **Soporte multiplataforma** (Windows, IoT, Cloud Instances).
-



Conclusión

Esta propuesta aborda un problema común en el mundo de la ciberseguridad: la dificultad de auditar y documentar configuraciones de forma eficiente.

La herramienta no solo **detecta vulnerabilidades**, sino que también **enseña y guía al usuario** para comprender y resolver los problemas, aportando valor tanto técnico como educativo.

Es un proyecto **útil, escalable y realista** para un TFG, con aplicación directa en auditorías de sistemas, bastionado y formación en seguridad.